

Data Communication Structures

Specialty Coffee Online Shop | Microservices and Containers (KM00DT91)

This document describes how the services communicate in the implemented application and the event schemas used. The order flow is **orchestrated** by the Ordering service; services talk over HTTP/REST, and logs and events are sent one-way to a central monitoring service.

1. Communication Patterns

Communication	Pattern	Justification
Browser → Gateway → services	Request-reply (REST)	The browser needs an immediate answer (product list, order status, login result), so synchronous HTTP request-reply through the gateway is the natural fit.
Gateway → Auth (verify)	Request-reply	Before forwarding an order, the gateway calls Auth to verify the token and waits for the valid / not-valid answer.
Ordering → Payment	Request-reply	The orchestrator must know the outcome (was payment captured?) before marking the order Paid, so it waits for the reply.
Services → Monitoring	One-way (events / logs)	Each service posts structured logs and the Ordering service posts an OrderPaid event to the monitoring service. Fire-and-forget, so a slow monitor never blocks the workflow.

Correlation: the *orderId* is carried through the order request, the payment call and reply, and the published event, so an action can be followed across services in the logs.

2. Event Schemas

Events and key messages are single-purpose, self-contained, and use JSON. Each carries the *orderId* plus metadata (event type, version, timestamp). Only the strictly needed fields are included.

OrderPlaced / capture request (Ordering → Payment)

```
{
  "orderId": "ORD-10231",
  "amount": { "value": 37.00, "currency": "EUR" }
}
```

PaymentCaptured (Payment → Ordering, the reply)

```
{
  "eventType": "PaymentCaptured",
  "version": 1,
  "timestamp": "2026-06-12T10:15:04Z",
  "orderId": "ORD-10231",
  "paymentId": "PAY-10231",
  "status": "captured"
}
```

OrderPaid (Ordering → Monitoring, the event)

```
{
  "eventType": "OrderPaid",
  "version": 1,
  "timestamp": "2026-06-12T10:15:05Z",
  "orderId": "ORD-10231",
  "paymentId": "PAY-10231"
}
```

Log entry (any service → Monitoring)

```
{
  "service": "ordering",
  "level": "INFO",
  "timestamp": "2026-06-12T10:15:05Z",
  "orderId": "ORD-10231",
  "message": "order paid"
}
```

The monitoring service aggregates these into simple metrics (orders paid, events, log entries, errors) exposed at /metrics and shown in the monitoring widget.